Proxmox VE Introduction

Day 1: Virtualization & Platform Fundamentals

Day 1 — Agenda

1. Course Introduction & Objectives
2. Virtualization Concepts — Type 1 vs. Type 2 Hypervisors
3. Proxmox VE Architecture Overview
4. Installation & Initial Configuration
5. The Proxmox Web UI & CLI Tools (`pvesh` , `qm` , `pct`)
6. Datacenter, Node & Cluster Concepts
7. User Management & Permissions (Realms, Roles)
8. Storage Overview — Local vs. Shared Storage
9. Networking Basics — Linux Bridges & VLANs
10. Lab Environment Walkthrough

1. Course Introduction & Objectives

By the end of this 3-day course, you will be able to:

- Explain how Proxmox VE fits into the modern virtualization landscape
- Install, configure, and operate a Proxmox VE node or cluster
- Create and manage Virtual Machines (QEMU/KVM) and Containers (LXC)
- Choose an appropriate storage backend for a given workload
- Configure backups with Proxmox Backup Server (PBS)
- Apply day-2 operational practices: monitoring, updates, troubleshooting

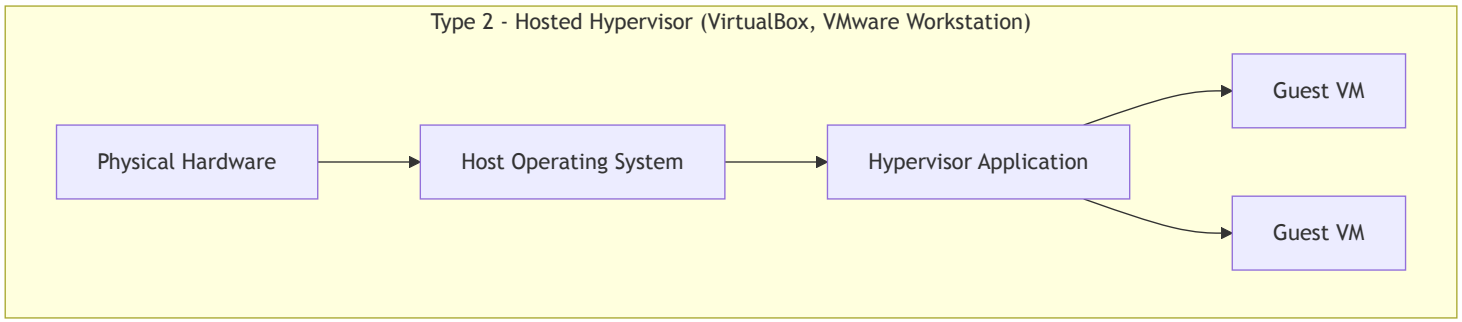
Target version for this course: Proxmox VE **9.2** (Debian Trixie base, kernel 7.0, QEMU 11.0, LXC 7.0, ZFS 2.4, Ceph Tentacle 20.2)

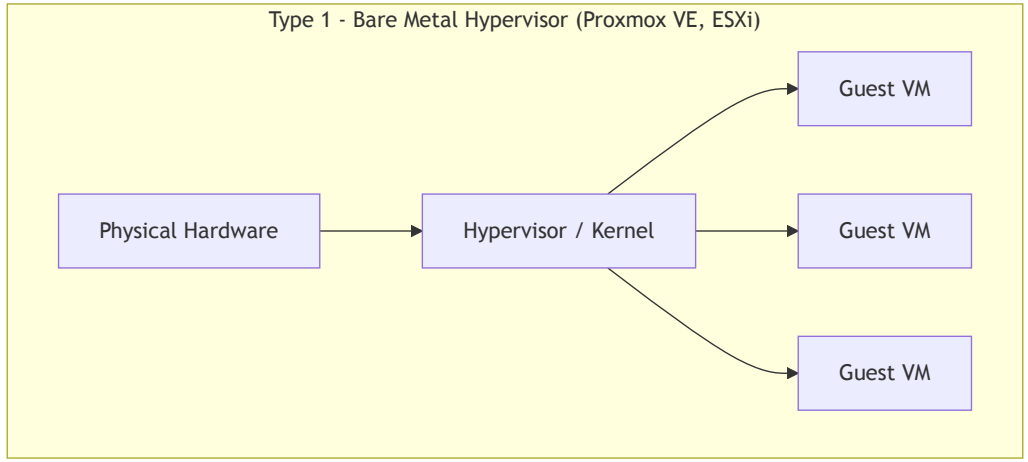
2. Virtualization Concepts

Virtualization lets a single physical machine run multiple isolated operating system instances ("guests") by abstracting CPU, memory, storage, and network.

Benefit	Description
Consolidation	Run many workloads on fewer physical servers
Isolation	Faults in one guest don't affect others
Portability	Guests can migrate between hosts
Elasticity	Resize CPU/RAM/disk without new hardware
DR/HA	Snapshot, replicate, and fail over guests

Two broad categories of hypervisor exist — this distinction matters because it explains **why** Proxmox VE can run guests with near bare-metal performance.


Type 2 - Hosted Hypervisor (VirtualBox, VMware Workstation)
Physical Hardware
Host Operating System
Hypervisor Application
Guest VM
Guest VM


Type 1 - Bare Metal Hypervisor (Proxmox VE, ESXi)
Physical Hardware
Hypervisor / Kernel
Guest VM
Guest VM
Guest VM

Hypervisor Comparison

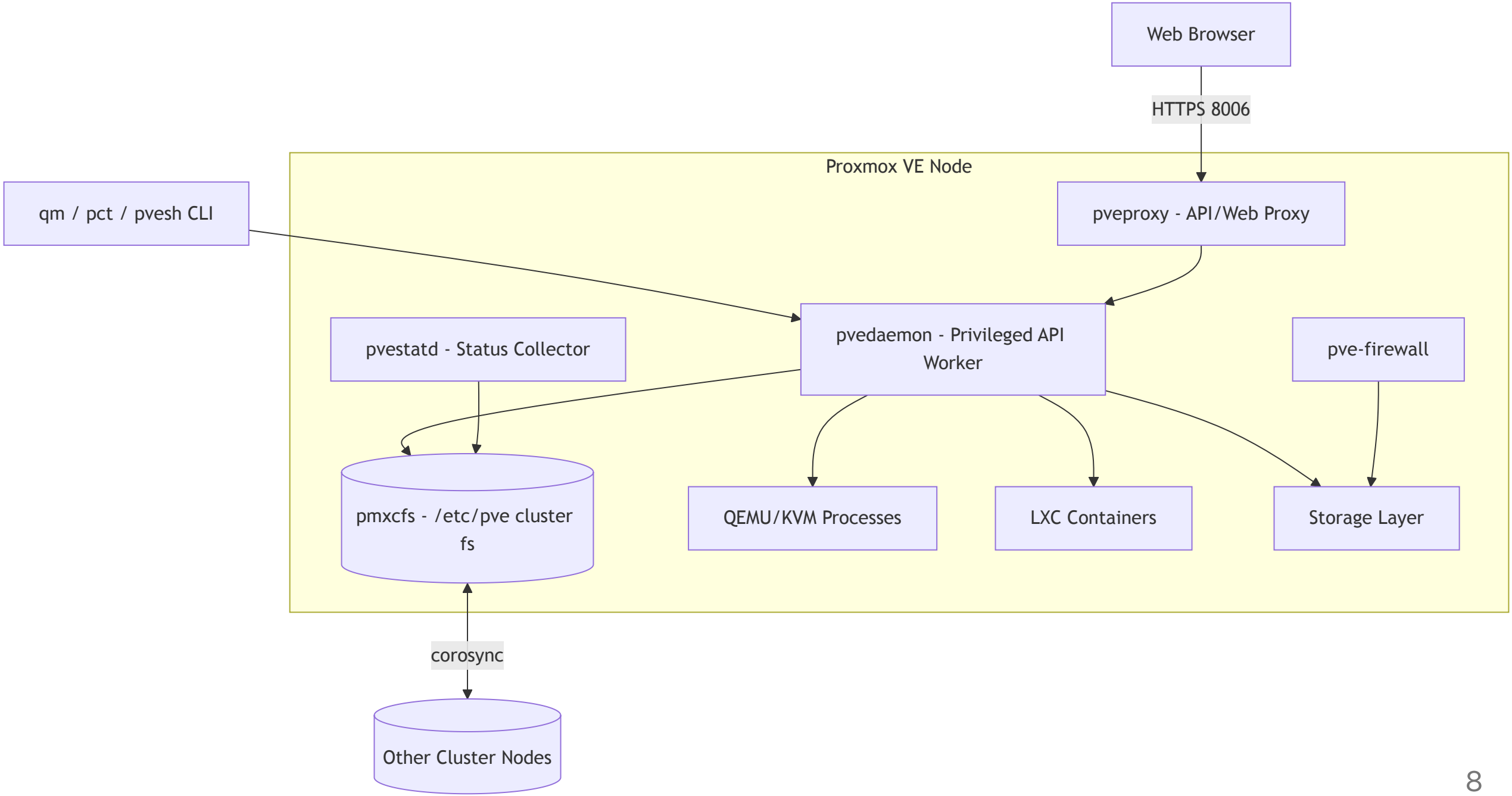
	Type 1 (Bare Metal)	Type 2 (Hosted)
Runs on	Hardware directly	Host operating system
Examples	**Proxmox VE**, VMware ESXi, Hyper-V	VirtualBox, VMware Workstation
Performance	Near-native	Overhead from host OS
Typical use	Datacenter, production servers	Desktop testing, developer sandboxes
Attack surface	Smaller (thin hypervisor layer)	Larger (full host OS + hypervisor app)

Proxmox VE is a **Type 1 (bare-metal) hypervisor platform** built on Debian Linux, using the Linux kernel's own **KVM** module for hardware virtualization plus **LXC** for OS-level

3. Proxmox VE Architecture Overview

Proxmox VE isn't a single monolithic program — it's a collection of cooperating services running on top of Debian Linux:

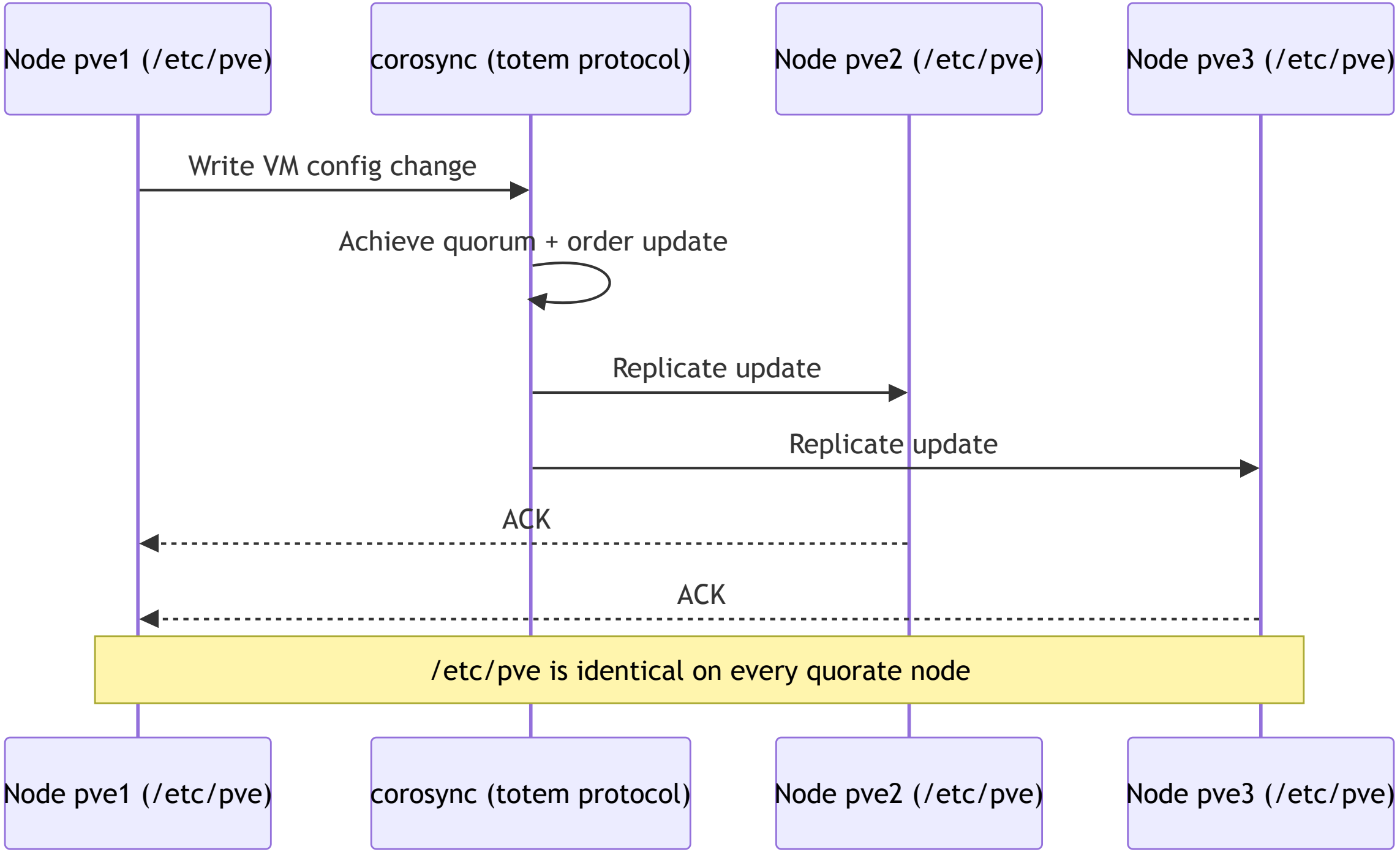
- **pveproxy** — HTTPS reverse proxy for the web UI/API (port 8006)
- **pvedaemon** — privileged worker that executes API requests as root
- **pmxcfs** — the cluster filesystem mounted at `/etc/pve`
- **pvestatd** — periodic statistics collector (CPU/RAM/disk/network)
- **pve-firewall** — datacenter/node/VM firewall enforcement
- **QEMU/KVM & LXC** — the actual guest virtualization engines


Web Browser
HTTPS 8006
Proxmox VE Node
qm / pct / pvesh CLI
pveproxy - API/Web Proxy
pvestatd - Status Collector
pvedaemon - Privileged API Worker
pve-firewall
pmxcfs - /etc/pve cluster fs
QEMU/KVM Processes
LXC Containers
Storage Layer
corosync
Other Cluster Nodes

The Cluster Filesystem — `pmxcfs`

Every node in a Proxmox VE cluster mounts a special filesystem at `/etc/pve`. It looks like ordinary files (`qemu-server/100.conf`, `storage.cfg`, `user.cfg`) but is actually backed by a **SQLite database replicated in real time** via `corosync`.

Because every quorate node shares an identical `/etc/pve`, you can manage the **whole cluster from any single node** — the web UI, `pvesh`, `qm`, and `pct` all just read/write this shared configuration store.

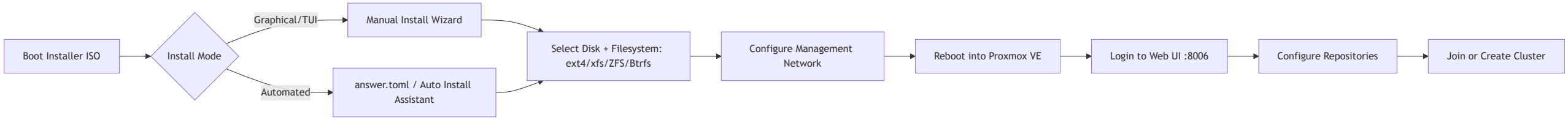

Node pve1 (/etc/pve)
corosync (totem protocol)
Node pve2 (/etc/pve)
Node pve3 (/etc/pve)
Write VM config change
Achieve quorum + order update
Replicate update
Replicate update
ACK
ACK
/etc/pve is identical on every quorate node
Node pve1 (/etc/pve)
corosync (totem protocol)
Node pve2 (/etc/pve)
Node pve3 (/etc/pve)

4. Installation & Initial Configuration

Proxmox VE ships as a single bootable ISO with two install paths:

Key installer decisions:

- **Filesystem**: `ext4` / `xfs` (simple), **ZFS** (snapshots, checksums, RAID), or Btrfs (technology preview)
- **Network**: static management IP is strongly recommended for production
- **Automated installs** use `proxmox-auto-install-assistant` with a TOML/JSON answer file — ideal for PXE-booted bare-metal provisioning at scale


Boot Installer ISO
Install Mode
Graphical/TUI
Manual Install Wizard
Automated
answer.toml / Auto Install Assistant
Select Disk + Filesystem: ext4/xfs/ZFS/Btrfs
Configure Management Network
Reboot into Proxmox VE
Login to Web UI :8006
Configure Repositories
Join or Create Cluster

First Boot Checklist

```
# Confirm the node is healthy
pveversion -v

# Point apt at a usable repository (no valid subscription yet)
echo "deb http://download.proxmox.com/debian/pve trixie pve-no-subscription" \
  > /etc/apt/sources.list.d/pve-no-subscription.list
rm -f /etc/apt/sources.list.d/pve-enterprise.list

apt update && apt full-upgrade -y

# Verify time sync (critical for corosync + certificates)
timedatectl status
```

Login to `https://<node-ip>:8006` with the `root@pam` account created during install.

5. Web UI & CLI Tools

Everything in the web UI is a thin client over the **same REST API** used by the CLI tools — nothing is UI-only.

Tool	Purpose
`pvesh`	Generic REST API client (`get` / `set` / `create` / `delete`)
`qm`	Manage QEMU/KVM virtual machines
`pct`	Manage LXC containers
`pvesm`	Manage storage
`pvecm`	Manage cluster membership
`pveceph`	Manage a hyper-converged Ceph cluster

```
# Same operation, three ways
qm list
pvesh get /nodes/pve1/qemu
curl -k -H "Authorization: PVEAPIToken=..." https://pve1:8006/api2/json/nodes/pve1/qemu
```

6. Datacenter, Node & Cluster Concepts

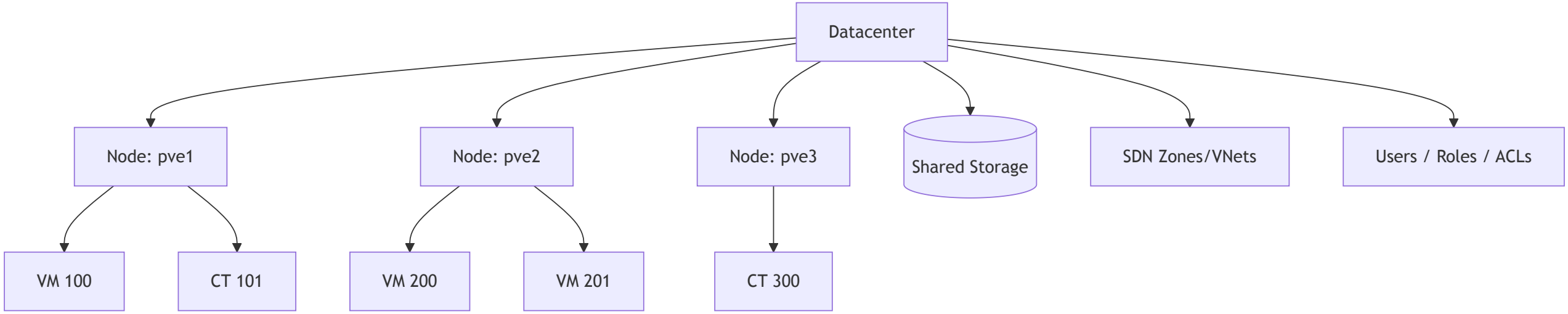
Proxmox VE organizes resources hierarchically. A single node is a fully functional standalone hypervisor; joining nodes into a **cluster** unlocks shared configuration, live migration, and HA.

```
# On the first node
pvecm create prod-cluster

# On each additional node
pvecm add pve1.example.com

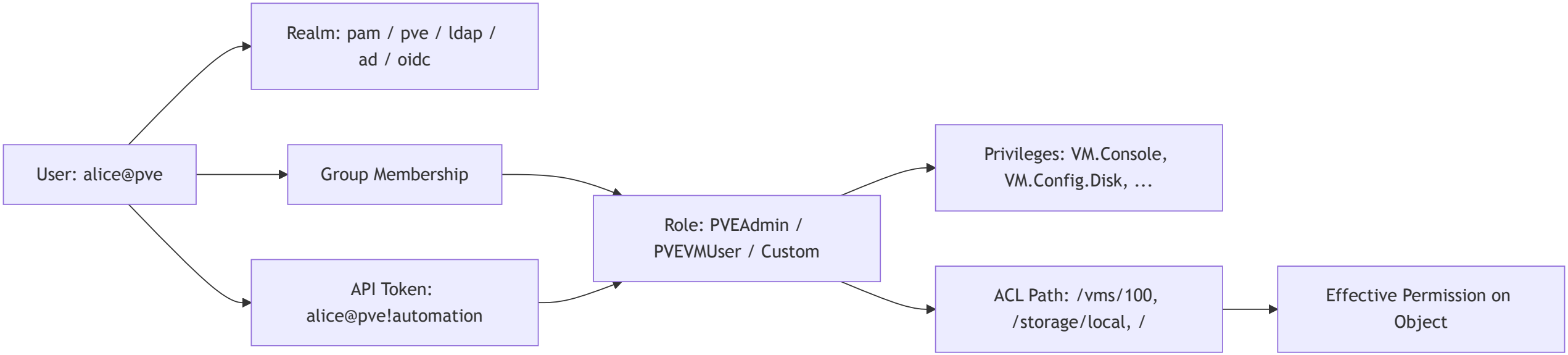
# Check quorum status
pvecm status
```

A cluster requires **quorum** (a majority of votes) via `corosync` before it will allow configuration changes — this prevents split-brain scenarios.


Datacenter
Node: pve1
Node: pve2
Node: pve3
Shared Storage
SDN Zones/VNets
Users / Roles / ACLs
VM 100
CT 101
VM 200
VM 201
CT 300

7. User Management & Permissions

Proxmox VE uses a layered permission model: **Users** belong to a **Realm** (authentication source), are placed in **Groups**, and are granted **Roles** on specific **ACL paths** in the resource tree.


Realm: pam / pve / ldap / ad / oidc
User: alice@pve
Group Membership
API Token: alice@pve!automation
Role: PVEAdmin / PVEVMUser / Custom
Privileges: VM.Console, VM.Config.Disk, ...
ACL Path: /vms/100, /storage/local, /
Effective Permission on Object

```
# Create a realm-backed user and grant a scoped role
pveum realm add ad-corp --type ad --server ad.example.com --domain example.com
pveum user add jsmith@ad-corp
pveum acl modify /vms/100 --users jsmith@ad-corp --roles PVEVMUser

# API tokens for automation (Terraform, Ansible, CI/CD)
pveum user token add automation@pve terraform --privsep 1
```

8. Storage Overview

Proxmox VE separates **storage configuration** (`/etc/pve/storage.cfg`, cluster-wide) from the **actual backend technology**.

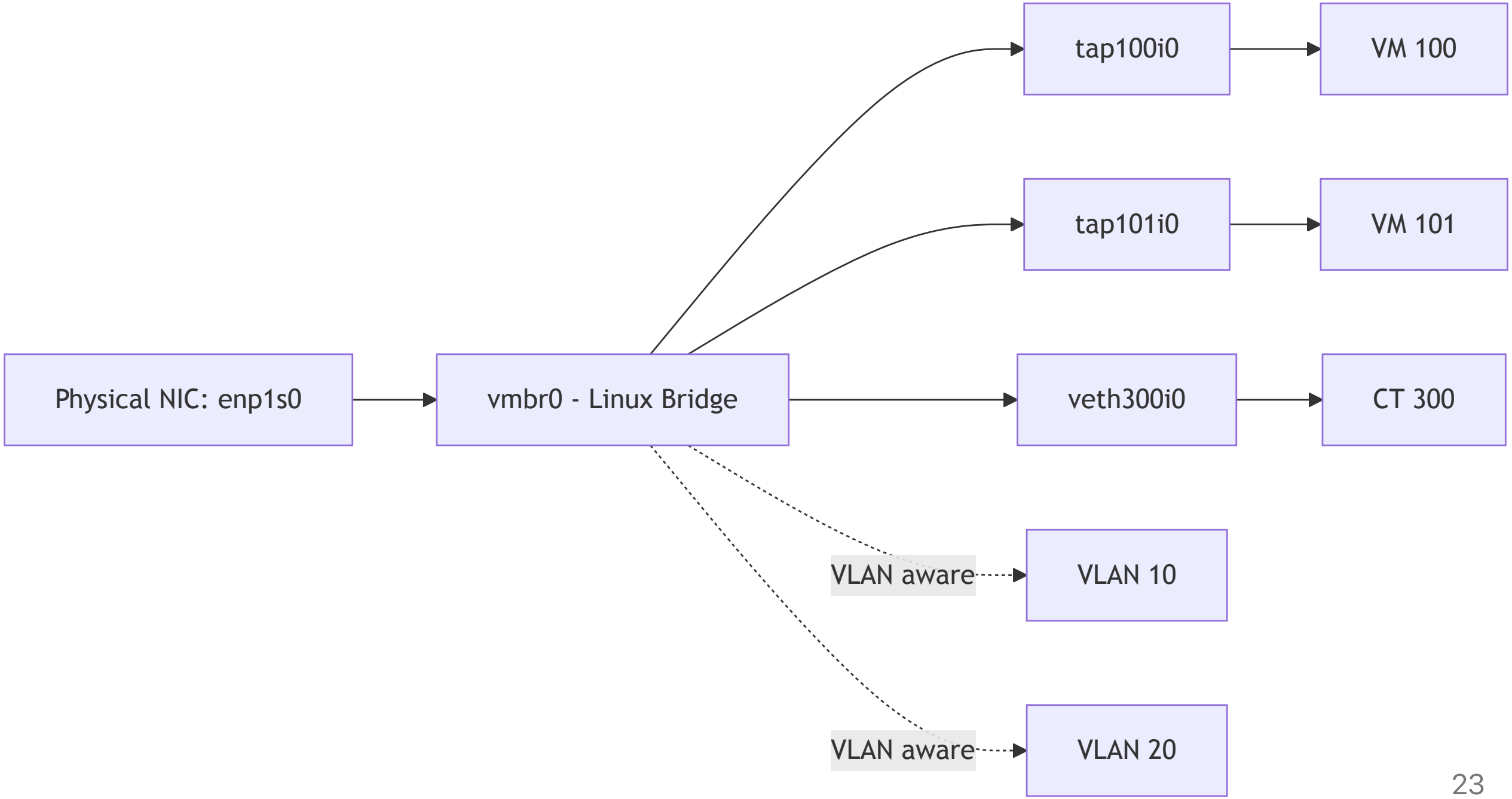
Storage type	Shared?	Snapshots	Typical use
Directory	No	qcow2 only	ISOs, simple single-node labs
LVM-thin	No	Yes	Fast local VM disks
ZFS	No (local)	Yes	Resilient single-node storage
NFS/CIFS	Yes	Depends on format	Shared ISO/backup storage
Ceph RBD/CephFS	Yes	Yes	HA clusters, scale-out
Proxmox Backup		N/A (backup	

Directory /

over iSCSI

9. Networking Basics

Proxmox VE networking builds on standard **Linux bridges** (`vmbr0`, `vmbr1`, …) — software switches that connect physical NICs, VM `tap` interfaces, and container `veth` pairs.


Physical NIC: enp1s0
vmbr0 - Linux Bridge
tap100i0
VM 100
tap101i0
VM 101
veth300i0
CT 300
VLAN aware
VLAN 10
VLAN aware
VLAN 20

```
# /etc/network/interfaces snippet
auto vmbr0
iface vmbr0 inet static
    address 10.10.10.5/24
    gateway 10.10.10.1
    bridge-ports enp1s0
    bridge-stp off
    bridge-fd 0
    bridge-vlan-aware yes
```

A **VLAN-aware bridge** lets a single `vmbr0` carry multiple tagged VLANs to different guests without needing one bridge per VLAN — this is the foundation for the SDN topics covered in the Architect course.

10. Lab Environment Walkthrough

For the exercises in this course you will use:

- A small Proxmox VE cluster (2–3 nodes) or nested-virtualization single node
- A shared storage target (NFS or ZFS) for later HA/backup exercises
- Network access to `download.proxmox.com` for updates and Ceph/PBS packages

Before Day 2: confirm you can log into the web UI, run `qm list`, and see at least one storage entry in `pvesm status`.

Day 1 — Key Takeaways

- Proxmox VE is a **Type 1 hypervisor platform** on Debian Linux, using KVM + LXC
- `/etc/pve` (**pmxcfs**) is the single source of truth, replicated cluster-wide
- Every web UI action maps to a **REST API** call — also reachable via `pvesh` / `qm` / `pct`
- Users, Realms, Roles, and ACL paths together define **who can do what, where**
- Storage and networking are both **pluggable** — the right backend depends on scale and HA requirements